

CreditSense: Methods Tried and Project Timeline

A readable record of the branches, ideas, detours, and score jumps that shaped the final project

Johnny Izhak Kyriakos | March 2026 working history

What this report is. The main report only explains the clean final public solution. This document does the opposite: it keeps the messy history intact enough to understand what was tried, what moved the score, what only looked promising for a moment, and why some branches were kept while others were left behind.

How the timeline was rebuilt. The reconstruction comes from saved score tables, result JSON files, long training logs, submission artifacts, and dated project chats. Some early intermediate outputs were overwritten later, so whenever an old exact number is missing, the report says so instead of pretending otherwise.

Reading guide

1. Timeline first: when the main turns happened.
2. Methods matrix second: what each lane was trying to do.
3. Compute notes third: what ran on Mac vs HPC and how long it took.
4. Final verdict last: what survived into the final project story.

One-line summary. The best gains came from stronger feature artifacts and careful blending of strong tabular branches. Pure novelty for its own sake usually lost.

1. Timeline At A Glance

The project changed direction a few times, but the main arc was fairly consistent: baseline recovery first, then stronger tree ensembles, then a target-stat branch, then neural or meta layers on top of those tabular sources, and finally a professor-inspired parallel lane to test whether broader feature generation could push the upstream score further.

If you only want the short version

- The first reliable jump came from turning the baseline into a stronger tree ensemble, not from chasing exotic models too early.
- The most important classical step was the target-stats branch.
- The best local score came later from a neural or meta blend on top of strong tabular branches, not from abandoning those branches.
- The professor lane improved the refreshed target-stats source a little, but the strongest final local blends still came from the already strong hybrid stack.

Main project turns and the score anchors they produced

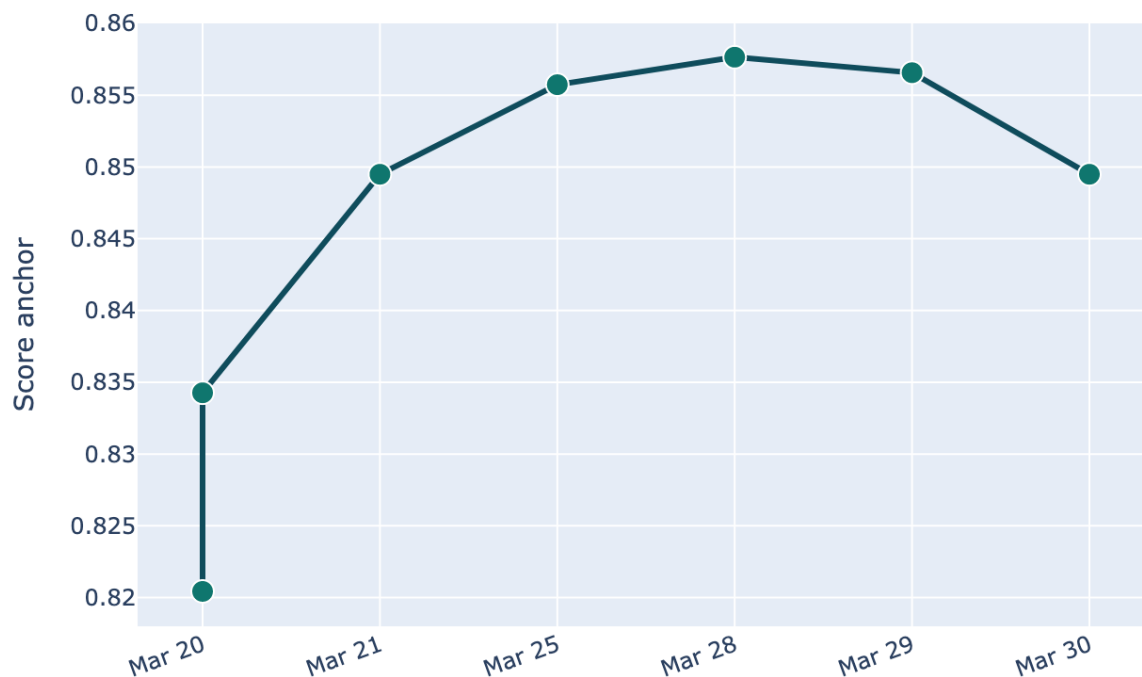


Figure 1: The main turns in the project, shown with the score anchor each phase produced.

Date	Phase	Score	What changed
Mar 20	Baseline reconstruction	0.82043 public	Recovered the assignment setup, rebuilt the baseline, and used the first Kaggle checks to calibrate the local split.
Mar 20	Poly-parallel tree blend	0.834266	The baseline turned into a broader tree stack, with an extra MPS-supported lane.
Mar 21	Overnight ensemble	0.849489	Tuned XGB, LGB, CatBoost, ExtraTrees, and MPS were combined into the first really systematic ensemble family.
Mar 25	Target-stats branch	0.855731	OOF target statistics and stronger pruning pushed the main classical branch into the mid 0.855 range.
Mar 28	Neural and meta topping	0.857645	The best local score came from a weighted neural/meta blend placed on top of strong tabular branches rather than a fully neural replacement.
Mar 29	Professor lane	0.856566	Broader feature generation and a refreshed target-stats branch improved the upstream source a bit, but did not replace the strongest final blends.
Mar 30	Public packaging	0.849489	The public repo was rebuilt around the cleaner overnight lane because it stayed reproducible and easy to explain.

Table 1: The seven project turns that matter most to the final story.

Two practical notes shaped the work from early on. First, Kaggle submissions were treated as scarce calibration

points rather than as blind search. Second, the repo became dense quickly, so each later stage was judged not only by score, but also by whether it stayed understandable enough to package into a clean final deliverable.

2. What Each Lane Was Trying To Do

Lane	Core idea	Why it was tried	Verdict
Early boosted trees	LightGBM, XGBoost, ExtraTrees, and simple blends on a shared holdout split	Fast way to beat the template and check whether the local validation scheme was trustworthy	It worked well enough to calibrate Kaggle quickly and became the foundation for the next families.
Poly parallel + MPS	Broader engineered features plus an extra MPS-supported branch	Test whether wider but still readable feature work could move the score before deeper refactors	It improved on the baseline and made branch-first thinking natural.
Overnight ensemble	Tuned XGB, LGB, CatBoost, ExtraTrees, MPS, and a learned combiner	Build one serious tabular family instead of many half-related scripts	This became the cleanest strong lane and later the public-facing path.
Target-stats family	OOB target statistics, stronger pruning, and richer tabular blending	Push the classical branch as far as possible before giving up on feature-side work	This became the main upstream source for the strongest local hybrids.
Neural and meta layers	Wide-deep, residual MLP, classifier-only blend, meta stack, and weighted blend	Check whether neural or meta layers could squeeze more value out of strong tabular predictions	Light meta layers helped. Fully replacing the tabular core usually did not.
Professor lane	Broader feature factory, stronger missingness features, classical screening, and GA-style refinement after pruning	Systematically test the professor's suggestions without throwing away the current winners	Useful as an upstream refinement lane, but not a clean replacement for the strongest final blends.

Table 2: The main branches were not random. Each lane was trying to answer a fairly specific question.

Saved best-score progression across the project

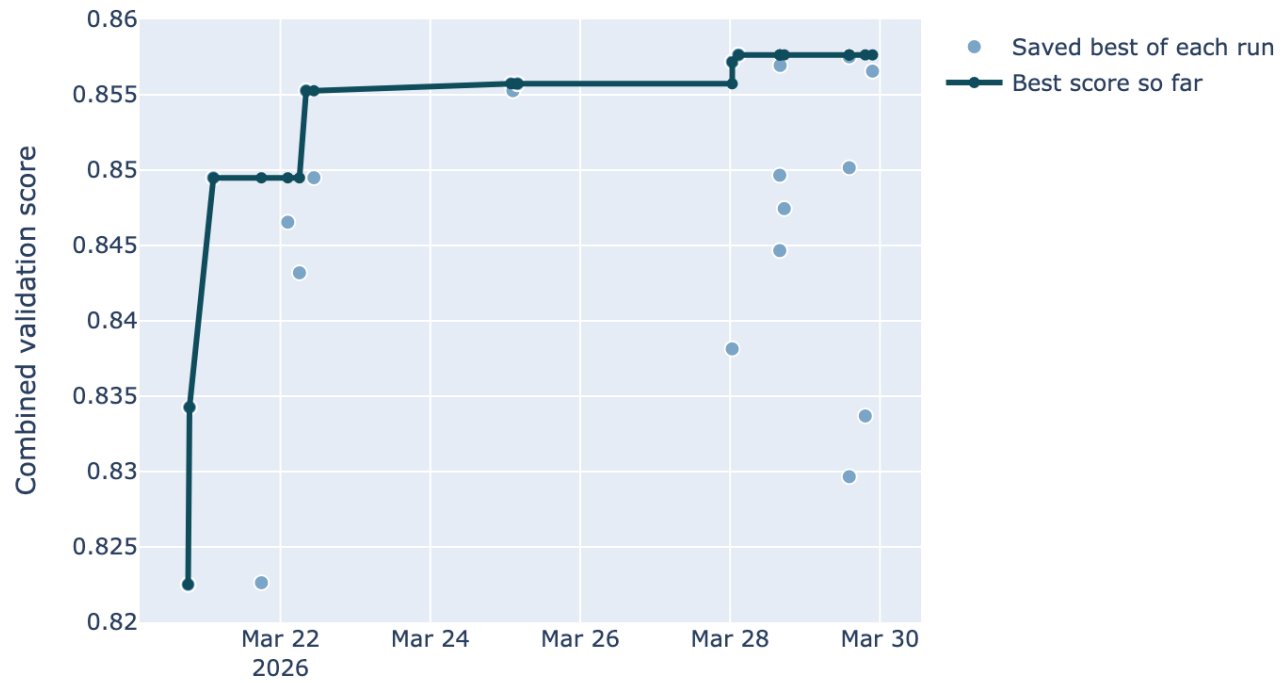


Figure 2: Saved best-score progression across the project. The general pattern was steady upward motion, but the gains became smaller and more expensive late in the project.

Top saved approaches by combined validation score

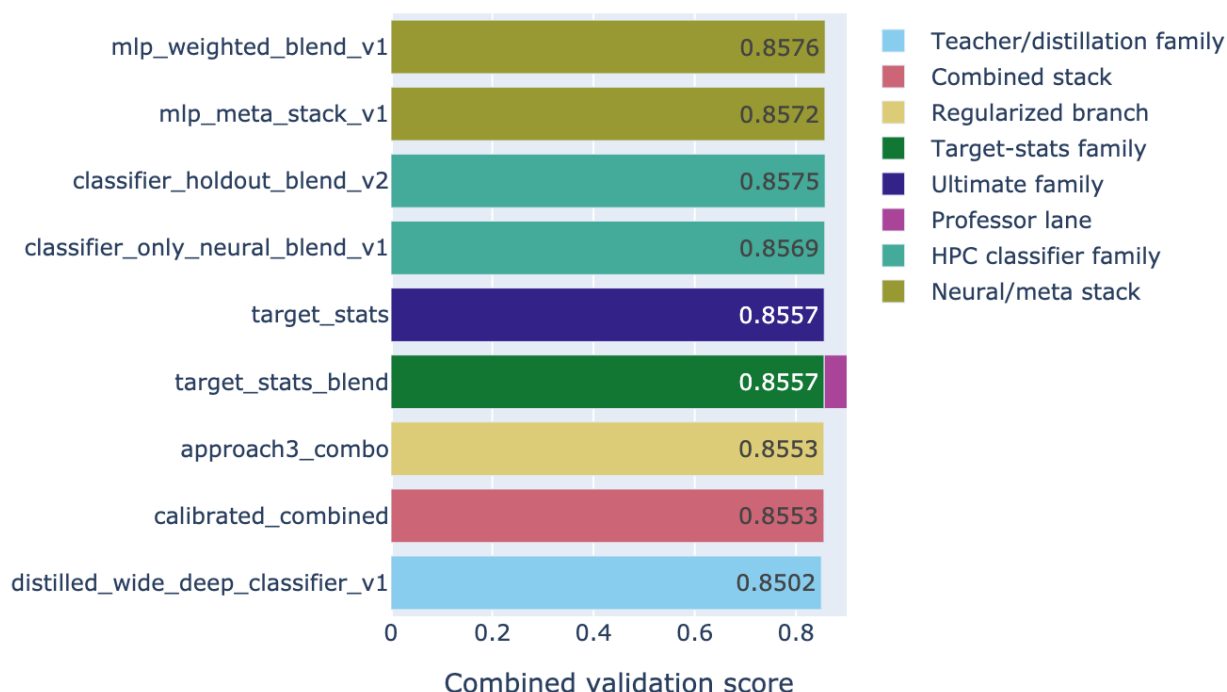


Figure 3: Top saved approaches by the combined validation metric. The strongest rows are not isolated single models; they are blends sitting on top of strong tabular sources.

Three takeaways kept repeating:

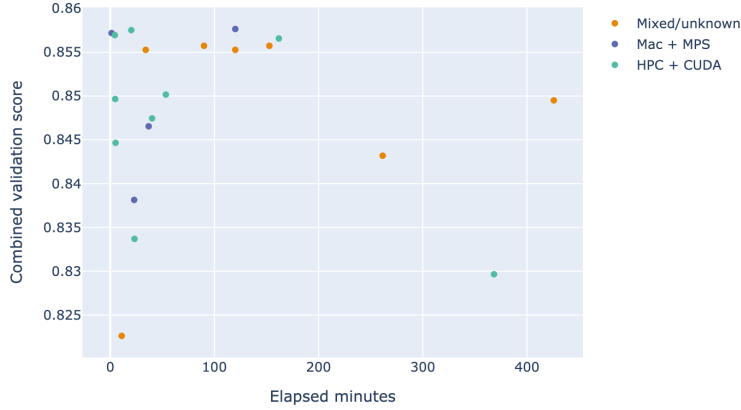
- Stronger feature artifacts mattered more than cosmetic model swapping.
- The best local scores came from combinations of good branches, not from one heroic standalone model.
- The project improved when it stayed selective. The most expensive feature or model idea was not automatically the best one.

This is why the project eventually settled on a layered story: build a reliable tabular source first, then add a lighter combination layer on top if it earns its keep.

3. Compute, Runtime, And Public Calibration

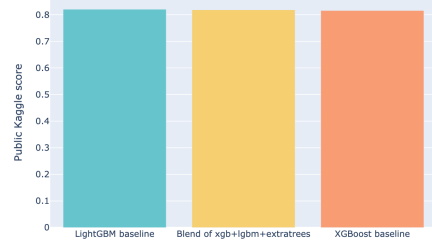
The project used both the MacBook and the MBZUAI HPC lab, but not for the same kind of work. Short MPS-supported experiments and compact meta models were comfortable on the Mac. Wider classifier families, cross-validated neural runs, and the professor lane made more sense on HPC, where the logs explicitly show workstations such as ws-15-031 and ws-15-032.

Runtime versus score for the saved experiment runs



(a) Runtime versus score for the saved runs.

First public Kaggle calibration checks that were explicitly recorded



(b) The early public Kaggle checks that were explicitly recorded.

Figure 4: The project needed both local speed and cluster muscle. Public Kaggle checks were used sparingly, mostly to calibrate the local metric rather than replace it.

Run	Score	Minutes	Compute note
mlp_meta_stack_v1	0.857181	1.33	Tiny Mac-side meta model; very cheap and surprisingly effective.
mlp_weighted_blend_v1	0.857645	120.06	Mac/MPS blend search; best local result that survived in the saved artifacts.
classifier_holdout_blend_v2	0.857517	20.31	HPC/CUDA run; strong late-stage competitor, especially on the classification side.
approach_professor_target_stats	0.856566	161.83	HPC refresh of the target-stats branch after the professor suggestions.
tree_teacher_classifier_bagging_v2	0.829660	368.29	Expensive and technically interesting, but not a practical winner on the project metric.

Table 3: A few runtime snapshots show the real trade-off: some of the best ideas were cheap, while some of the most expensive ones were mainly educational.

The public leaderboard was helpful, but only in small doses. The first Kaggle calibration on March 20 showed that the local ranking was useful: LightGBM was best publicly among the first three, the blend was close behind, and XGBoost trailed slightly. That was enough to justify keeping Kaggle as a checkpoint rather than turning the whole project into a submission lottery.

Later public scores were not kept with the same level of detail in the retained artifacts, so the second half of the project had to lean mostly on saved validation tables and training logs.

4. What Survived, What Stayed As Exploration

What clearly survived

- strong tabular feature artifacts, especially the target-stats substrate
- careful branch combination instead of single-model heroics
- lighter neural or meta layers used as finishers, not as total replacements
- splitting work sensibly across Mac/MPS and HPC/CUDA
- treating Kaggle submissions as calibration points rather than blind search

What mostly stayed as exploration

- fully replacing the strong tabular core with pure classifier-only neural ideas
- the most aggressive feature factories when they added more complexity than signal
- tree-teacher and distillation-style branches as final leaders
- the professor classical screen as a direct replacement for the best existing lanes
- any branch whose exact history was not preserved well enough to defend with artifacts

The cleanest way to say it is this: the project got stronger when it stayed disciplined. The useful moves were not random flashes of novelty. They were usually the boring but effective ones: better feature artifacts, cleaner pruning, stronger tabular branches, and then a careful blend on top.

The professor-inspired lane still mattered. It widened the search space, improved the refreshed target-stats branch slightly, and forced the project to think more carefully about missingness and feature selection. But it behaved more like a good upstream research lane than like a full new winner.

That is also why the public repo was finally rebuilt around the overnight family instead of the absolute best local blend. The overnight path was strong, explicit, reproducible, and easy to explain. The later local winners were real, but they lived in a denser and more experimental part of the project history.

Final takeaways

1. The score story was mostly a story about better artifacts and better combinations, not about abandoning trees.
2. The best local branch did beat the public lane, but the public lane was much easier to package cleanly.
3. Some old exact results are gone, but the overall trend is still clear from the saved score tables and logs.
4. If the project were continued, the most sensible next step would be to keep the target-stats backbone, keep the professor lane as an upstream research branch, and only retune the final combiner when the upstream source truly improves.